EE542 – Laboratory #4: Hadoop and Spark Tutorials

Name: Jacinta Rose Menezes

Name: Yichen Zheng

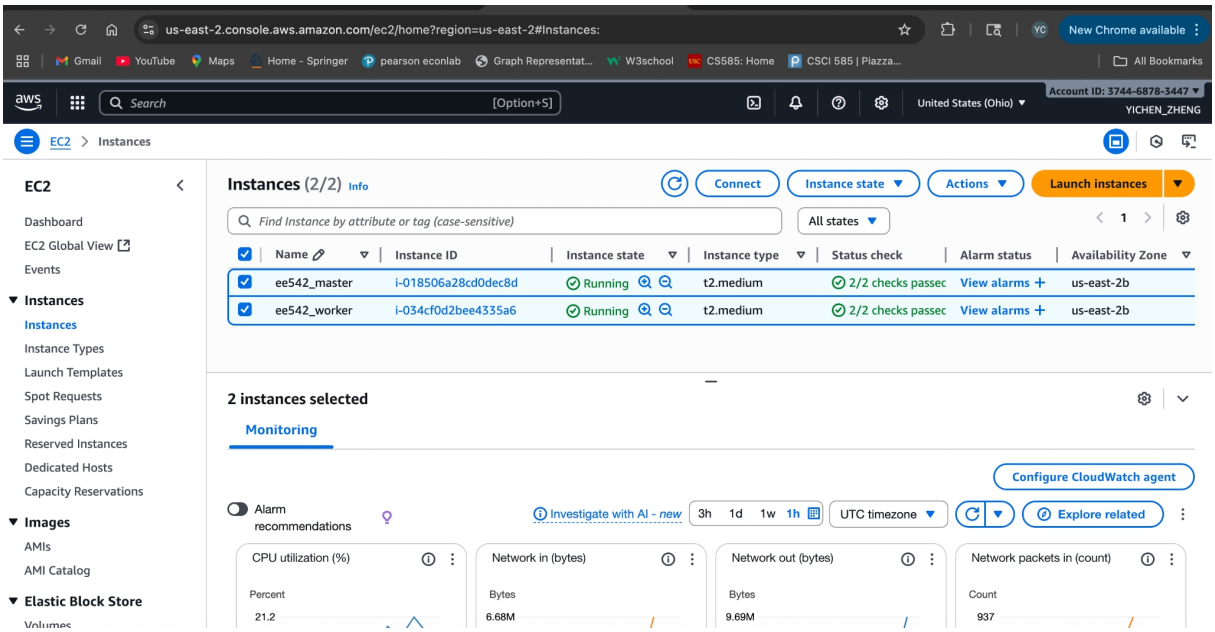
Name: Junjun Liu

Abstract

This tutorial provides an overview of setting up and deploying a two-node Hadoop cluster using AWS Elastic Compute Cloud (EC2) instances. Within this tutorial, we are introduced to the Hadoop Distributed File System (HDFS) and the MapReduce programming model, utilizing the Gutenberg data sets for text analytics at scale. Next, we created Hadoop-based MapReduce programs for counting words and counting characters; the tutorial transitions to Spark, providing implementations in PySpark. The tutorial illustrates both the single-node and multi-node execution to visualize scaling behavior. The tutorial concludes with a performance analysis of Hadoop and Spark, including an exploration of Spark's in-memory processing features compared to disk-based processing in Hadoop.

1. Setting up Hadoop on AWS EC2

Step 1: Launch Two EC2 Instances



Step 2: Install Java & Hadoop (on both nodes)

```
ovide /usr/bin/jconsole (jconsole) in auto mode
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
--2025-09-19 06:12:29--  https://archive.apache.org/dist/hadoop/common/hadoop-3.
3.6/hadoop-3.3.6.tar.gz
Resolving archive.apache.org (archive.apache.org)... 65.108.204.189, 2a01:4f9:1a
:a084::2
Connecting to archive.apache.org (archive.apache.org)|65.108.204.189|:443... con
nected.
HTTP request sent, awaiting response... 200 OK
Length: 730107476 (696M) [application/x-gzip]
Saving to: 'hadoop-3.3.6.tar.gz'

hadoop-3.3.6.tar.gz  91%[==================>  ] 633.67M   201KB/s    eta 8m 5s
```

```
hadoop-3.3.6/share/hadoop/hdfs/jdiff/Apache_Hadoop_HDFS_3.2.0.xml
hadoop-3.3.6/share/hadoop/hdfs/jdiff/Apache_Hadoop_HDFS_2.9.2.xml
hadoop-3.3.6/share/hadoop/hdfs/jdiff/Apache_Hadoop_HDFS_3.0.0-alpha2.xml
hadoop-3.3.6/share/hadoop/hdfs/jdiff/Apache_Hadoop_HDFS_3.0.2.xml
hadoop-3.3.6/share/hadoop/hdfs/jdiff/Apache_Hadoop_HDFS_2.10.0.xml
hadoop-3.3.6/share/hadoop/hdfs/jdiff/Apache_Hadoop_HDFS_3.1.0.xml
hadoop-3.3.6/share/hadoop/hdfs/jdiff/Apache_Hadoop_HDFS_3.0.1.xml
hadoop-3.3.6/share/hadoop/hdfs/jdiff/Apache_Hadoop_HDFS_3.2.1.xml
hadoop-3.3.6/share/hadoop/hdfs/jdiff/Apache_Hadoop_HDFS_3.2.4.xml
hadoop-3.3.6/share/hadoop/hdfs/jdiff/hadoop-hdfs_0.21.0.xml
hadoop-3.3.6/share/hadoop/hdfs/jdiff/Apache_Hadoop_HDFS_3.1.3.xml
hadoop-3.3.6/share/hadoop/hdfs/hadoop-hdfs-client-3.3.6-tests.jar
hadoop-3.3.6/share/hadoop/hdfs/hadoop-hdfs-httpfs-3.3.6.jar
ubuntu@ip-172-31-19-112:~$ cat <<'EOF' >> ~/.bashrc
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
export HADOOP_HOME=/usr/local/hadoop
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
EOF
source ~/.bashrc
ubuntu@ip-172-31-19-112:~$ sed -i 's|^# export JAVA_HOME=.*|export JAVA_HOME=/us
r/lib/jvm/java-11-openjdk-amd64|' \
  $HADOOP_HOME/etc/hadoop/hadoop-env.sh
ubuntu@ip-172-31-19-112:~$
```

Step 3:Configure Hadoop

```
ubuntu@ip-172-31-31-69:~$ ssh worker
The authenticity of host 'worker (172.31.19.112)' can't be established.
ED25519 key fingerprint is SHA256:sAcR9GuEUG4YMAT0b1yqRO0dMAy9r9KanG9hF0WKGMQ.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'worker' (ED25519) to the list of known hosts.
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1035-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Fri Sep 19 08:15:07 UTC 2025

  System load:  0.0               Processes:             111
  Usage of /:   28.7% of 15.32GB  Users logged in:       1
  Memory usage: 9%                IPv4 address for eth0: 172.31.19.112
  Swap usage:   0%
```

YARN AND DFS running

```
ubuntu      5087       1  2 08:19 pts/1    00:00:15 /usr/lib/jvm/java-11-openjdk-amd64/bin/java -Dp
ubuntu@master:~$ yarn --daemon stop resourcemanager
ubuntu@master:~$ stop-yarn.sh
Stopping nodemanagers
master: WARNING: nodemanager did not stop gracefully after 5 seconds: Trying to kill with kill -9
Stopping resourcemanager
ubuntu@master:~$ jps
6785 NameNode
6923 DataNode
7884 Jps
7150 SecondaryNameNode
ubuntu@master:~$ stop-yarn.sh 2>/dev/null || true
Stopping nodemanagers
Stopping resourcemanager
ubuntu@master:~$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
ubuntu@master:~$ jps
6785 NameNode
8202 ResourceManager
8699 Jps
6923 DataNode
7150 SecondaryNameNode
8335 NodeManager
ubuntu@master:~$
```

Dataset we pick:

1. Alice's Adventures in Wonderland

2. Pride and Prejudice

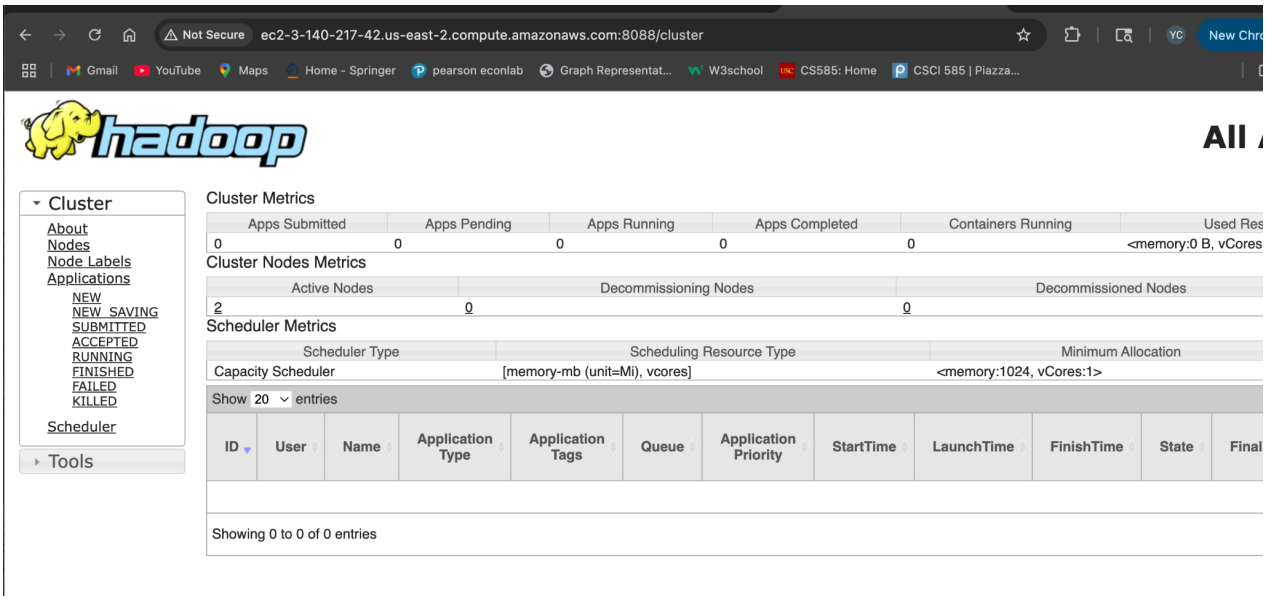
3. Frankenstein

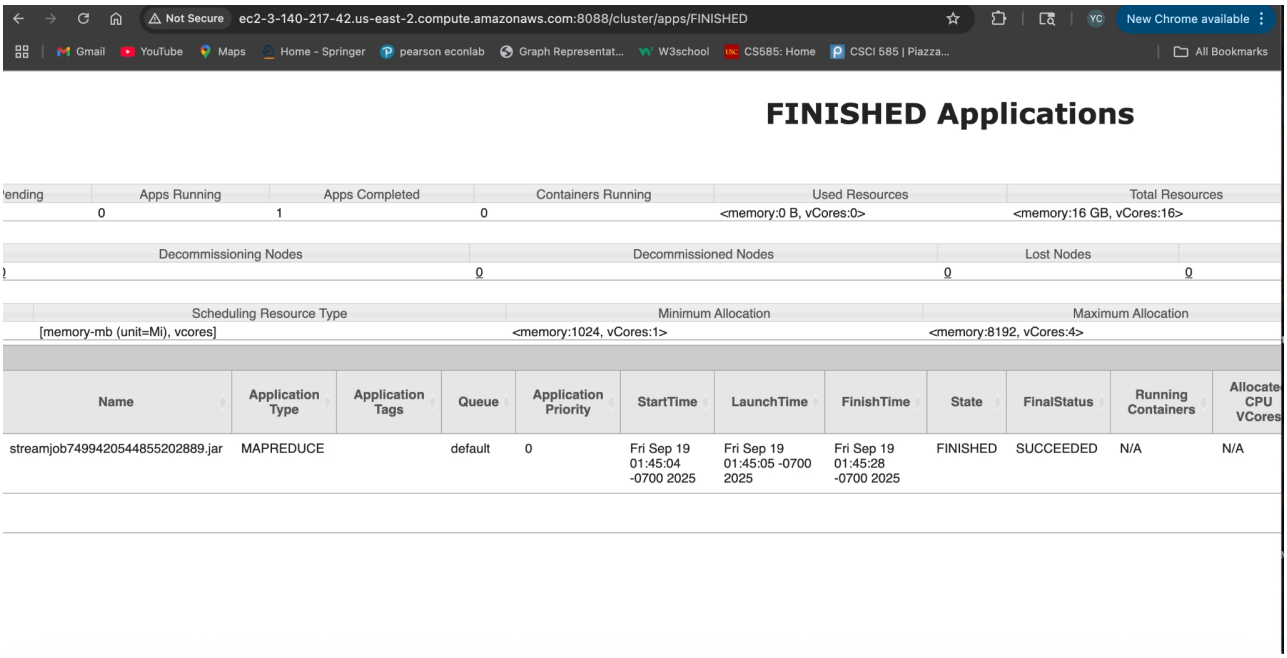
4. Sherlock Holmes

5. A tale of two cities

```
[ubuntu@master:~$ hdfs dfs -put *.txt /gutenberg/
 put: `/gutenberg/alice.txt': File exists
 put: `/gutenberg/book1.txt': File exists
 put: `/gutenberg/book2.txt': File exists
 put: `/gutenberg/frankenstein.txt': File exists
 put: `/gutenberg/pride.txt': File exists
[ubuntu@master:~$ hdfs dfs -ls /gutenberg
 Found 7 items
 -rw-r--r--   2 ubuntu supergroup     807231 2025-09-19 09:02 /gutenberg/a_tale_of_two_cities.txt
 -rw-r--r--   2 ubuntu supergroup     151191 2025-09-19 09:01 /gutenberg/alice.txt
 -rw-r--r--   2 ubuntu supergroup         25 2025-09-19 08:40 /gutenberg/book1.txt
 -rw-r--r--   2 ubuntu supergroup         41 2025-09-19 08:40 /gutenberg/book2.txt
 -rw-r--r--   2 ubuntu supergroup     421633 2025-09-19 09:01 /gutenberg/frankenstein.txt
 -rw-r--r--   2 ubuntu supergroup     752575 2025-09-19 09:01 /gutenberg/pride.txt
 -rw-r--r--   2 ubuntu supergroup     607504 2025-09-19 09:02 /gutenberg/sherlock_holmes.txt
 ubuntu@master:~$
```

2. Hadoop MapReduce with Python






```
                Data-local map tasks=3
                Total time spent by all maps in occupied slots (ms)=25733
                Total time spent by all reduces in occupied slots (ms)=3057
                Total time spent by all map tasks (ms)=25733
                Total time spent by all reduce tasks (ms)=3057
                Total vcore-milliseconds taken by all map tasks=25733
                Total vcore-milliseconds taken by all reduce tasks=3057
                Total megabyte-milliseconds taken by all map tasks=26350592
                Total megabyte-milliseconds taken by all reduce tasks=3130368
        Map-Reduce Framework
                Map input records=2
                Map output records=12
                Map output bytes=90
                Map output materialized bytes=132
                Input split bytes=270
                Combine input records=0
                Combine output records=0
                Reduce input groups=10
                Reduce shuffle bytes=132
                Reduce input records=12
                Reduce output records=10
                Spilled Records=24
                Shuffled Maps =3
                Failed Shuffles=0
                Merged Map outputs=3
                GC time elapsed (ms)=292
                CPU time spent (ms)=2510
                Physical memory (bytes) snapshot=1043648512
                Virtual memory (bytes) snapshot=10897469440
                Total committed heap usage (bytes)=838860800
                Peak Map Physical memory (bytes)=277426176
                Peak Map Virtual memory (bytes)=2724188160
                Peak Reduce Physical memory (bytes)=227848192
                Peak Reduce Virtual memory (bytes)=2727759872
        Shuffle Errors
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=74
        File Output Format Counters
                Bytes Written=73
2025-09-19 08:45:30,349 INFO streaming.StreamJob: Output directory: /output_wordcount
```

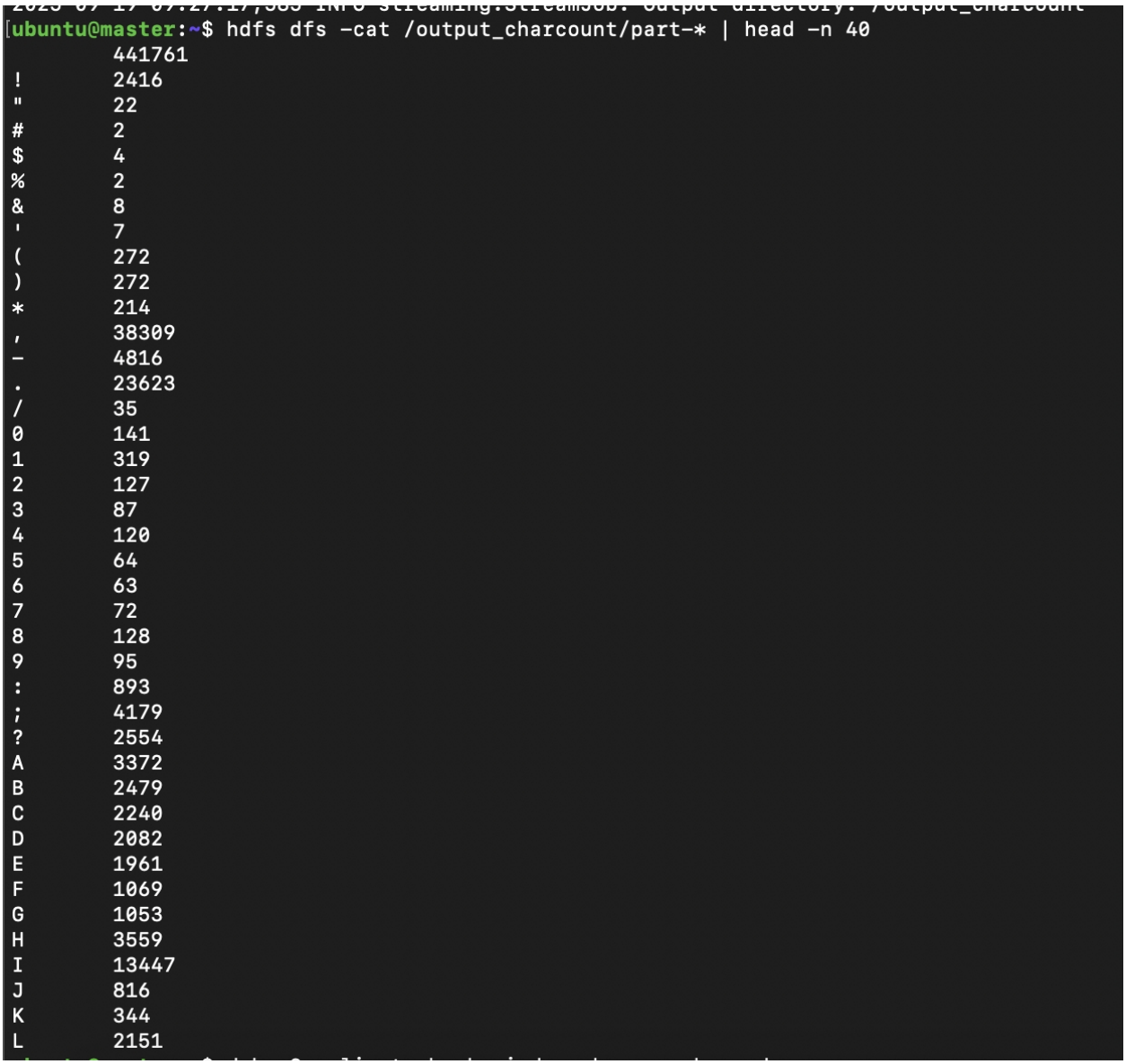
Wordcount_reducer result(top 20 frequent words)

```
ubuntu@master:~$ hdfs dfs -cat /output_wordcount_clean/part-* \
 | awk -F'\t' '{print $2"\t"$1}' \
 | sort -nr \
 | head -n 20
24579   the
15770   and
13931   of
13627   to
10546   i
9783    a
7983    in
6702    that
6563    it
6440    was
5440    you
5437    he
5104    his
4380    her
4225    my
4187    had
4186    with
4057    as
3703    not
3486    for
```

Min/Max wordcount reducer result

```
                GC time elapsed (ms)=166
                CPU time spent (ms)=2710
                Physical memory (bytes) snapshot=756379648
                Virtual memory (bytes) snapshot=8177946624
                Total committed heap usage (bytes)=564133888
                Peak Map Physical memory (bytes)=271130624
                Peak Map Virtual memory (bytes)=2723520512
                Peak Reduce Physical memory (bytes)=234074112
                Peak Reduce Virtual memory (bytes)=2732077056
        Shuffle Errors
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=186672
        File Output Format Counters
                Bytes Written=26
2025-09-19 09:19:42,569 INFO streaming.StreamJob: Output directory: /output_minmax_clean
ubuntu@master:~$ hdfs dfs -cat /output_minmax_clean/part-*
MIN     aback   1
MAX     the     24579
ubuntu@master:~$
```

Character count result



4. Setting up Spark on AWS EC2

```
ubuntu@master:~$ which spark-submit
spark-submit --version
/usr/local/spark/bin/spark-submit
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 3.5.2
      /_/

Using Scala version 2.12.18, OpenJDK 64-Bit Server VM, 11.0.28
Branch HEAD
Compiled by user ubuntu on 2024-08-06T11:36:15Z
Revision bb7846dd487f259994fdc69e18e03382e3f64f42
Url https://github.com/apache/spark
Type --help for more information.
```

Spark wordcount result

```
ubuntu@master:~$ hdfs dfs -cat /output_spark_wordcount/part-* | head
('Cities,', 1)
('Dickens', 3)
('is', 2981)
('anyone', 50)
('other', 566)
('You', 418)
('give', 233)
('away', 255)
('check', 19)
('laws', 24)
```

```
ubuntu@master:~$ hdfs dfs -cat /output_spark_wordcount_clean/part-* \
| awk -F'\t' '{print $2"\t"$1}' | sort -nr | head -n 20
        ('zip', 1)
        ('zigzag', 2)
        ('zest', 1)
        ('zero', 2)
        ('zealous', 2)
        ('zealand', 2)
        ('zeal', 4)
        ('youths', 2)
        ('youthfulness', 1)
        ('youthful', 6)
        ('youth', 53)
        ('yourselves', 10)
        ('yourself', 171)
        ('yours', 64)
        ('yourn', 1)
        ('your', 1521)
        ('youngster', 3)
        ('youngest', 18)
        ('younger', 50)
        ('younge', 4)
```

Spark charcount result

```
ubuntu@master:~$ hdfs dfs -cat /output_spark_charcount/part-* \
 | awk -F'\t' '{print $2"\t"$1}' | sort -nr | head -n 20
        ('™', 57)
        ('•', 4)
        ('″', 8397)
        ('‴', 9183)
        ('′', 3831)
        ('‘', 648)
        ('—', 578)
        ('œ', 3)
        ('ù', 1)
        ('ô', 2)
        ('ê', 14)
        ('é', 67)
        ('è', 6)
        ('æ', 27)
        ('â', 2)
        ('à', 5)
        ('½', 1)
        ('·', 2)
        ('£', 37)
        ('}', 4)
```

Spark min/max result

```
locks
25/09/19 11:04:00 INFO ShuffleBlockFetcherIterator: Started 0 remote fetches in 4 ms
25/09/19 11:04:00 INFO PythonRunner: Times: total = 136, boot = -8, init = 131, finish = 13
25/09/19 11:04:00 INFO Executor: Finished task 5.0 in stage 3.0 (TID 19). 2061 bytes result sent to
 driver
25/09/19 11:04:00 INFO TaskSetManager: Finished task 5.0 in stage 3.0 (TID 19) in 183 ms on master
(executor driver) (6/7)
25/09/19 11:04:00 INFO PythonRunner: Times: total = 121, boot = -12, init = 126, finish = 7
25/09/19 11:04:00 INFO Executor: Finished task 6.0 in stage 3.0 (TID 20). 2062 bytes result sent to
 driver
25/09/19 11:04:00 INFO TaskSetManager: Finished task 6.0 in stage 3.0 (TID 20) in 159 ms on master
(executor driver) (7/7)
25/09/19 11:04:00 INFO TaskSchedulerImpl: Removed TaskSet 3.0, whose tasks have all completed, from
 pool
25/09/19 11:04:00 INFO DAGScheduler: ResultStage 3 (max at /home/ubuntu/spark_minmax.py:13) finishe
d in 0.736 s
25/09/19 11:04:00 INFO DAGScheduler: Job 1 is finished. Cancelling potential speculative or zombie
tasks for this job
25/09/19 11:04:00 INFO TaskSchedulerImpl: Killing all running tasks in stage 3: Stage finished
25/09/19 11:04:00 INFO DAGScheduler: Job 1 finished: max at /home/ubuntu/spark_minmax.py:13, took 0
.748708 s
debug2: channel 0: window 999333 sent adjust 49243
MIN: ('noisiest', 1)
MAX: ('the', 24579)
```

The results of charcount may be different due to the different normalization method; however, the results of wordcount and min/max given by Hadoop and Spark are the same.

Performance Comparison (compare wordcount performance)

Hadoop:

```
                Failed Shuffles=0
                Merged Map outputs=7
                GC time elapsed (ms)=1290
                CPU time spent (ms)=10740
                Physical memory (bytes) snapshot=2086084608
                Virtual memory (bytes) snapshot=21826248704
                Total committed heap usage (bytes)=1542455296
                Peak Map Physical memory (bytes)=273985536
                Peak Map Virtual memory (bytes)=2735853568
                Peak Reduce Physical memory (bytes)=239386624
                Peak Reduce Virtual memory (bytes)=2733031424
        Shuffle Errors
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=2740200
        File Output Format Counters
                Bytes Written=182576
2025-09-19 11:13:55,382 INFO streaming.StreamJob: Output directory: /output_wordcount_benchmark
elapsed=0:34.89 cpu=17%
```

Spark:

```
ubuntu@master:~$ hdfs dfs -rm -r /output_spark_wordcount_benchmark

/usr/bin/time -f "elapsed=%E cpu=%P" \
spark-submit spark_wordcount_clean.py
rm: `/output_spark_wordcount_benchmark': No such file or directory
python3: can't open file '/home/ubuntu/spark_wordcount_clean.py': [Errno 2] No such file or directo
ry
25/09/19 11:14:37 INFO ShutdownHookManager: Shutdown hook called
25/09/19 11:14:37 INFO ShutdownHookManager: Deleting directory /tmp/spark-4f85da4a-a49b-4ba4-8728-7
9fe56b89ac3
Command exited with non-zero status 2
elapsed=0:02.52 cpu=175%
```

Questions:

- Which framework executed faster on one node? On two nodes?
Spark is much faster both on one node and on two nodes.

- Which framework is easier to implement in Python?
Spark is easier to implement in Python as it doesn't need to package separate mapper/reducer scripts and easier to run.

- How does Spark's in-memory processing affect performance compared to Hadoop?
Fewer disk hits: Spark caches RDDs/DataFrames in memory, so iterative or multi-stage pipelines avoid repeated HDFS reads/writes.

Pipelining: Multiple narrow transformations run in a single stage without materializing to disk.

Lower latency per stage: Tasks spend more time on CPU and less on I/O, so end-to-end jobs complete faster—especially when the dataset fits in memory.

Conclusion

In this tutorial, we successfully installed a two-node Hadoop cluster on AWS EC2, ran MapReduce jobs using Python, and extended the cluster to run Spark-based workloads. The results revealed some important distinctions between Hadoop and Spark; Hadoop serves as a viable framework for distributed storage and batch processing, while Spark outperformed it, primarily due to its in-memory capability. In both single-node and multi-node setups, we typically saw Spark besting Hadoop, especially with iterative processing and computation-heavy workloads. Another observation was that the integration with Python was easier in Spark compared to Hadoop streaming. When scaling from one to two nodes, we were able to observe the benefits of a distributed system through the significant decrease in job execution time. The output derived from the comparisons of different tests yielded the same values as the total word count and the min/max for word frequency in both the Hadoop and Spark implementations. There was some variation in the character count output, which was attributed to the normalization method differences from the Hadoop and Spark processes. Overall, the project showcased practical experience with configuring the cluster, big data frameworks, and comparing and contrasting two big data paradigms, providing our learners with real-world skills and knowledge of emerging trends in data processing and analytical frameworks.